



School of Future Tech

Music Streaming Backend

Backend Project Report

Daksh Ranjan Srivastava

Cohort - Jeff Bezos

Roll no - 150096725087

Subject - Backend, [Node.js](#), Firebase, MongoDB

Case Study 88: Backend Development

Comprehensive Technical Project Report: TuneWave Music Streaming Platform

Course: B.Tech Computer Science and Engineering

Subject: Backend Development – Node.js, Express.js & MongoDB

Project Title: TuneWave: High-Fidelity Music Streaming Service

Author / Developer: Daksh Srivastava

Repository: https://github.com/Daksh-create349/TuneWave-Backend_Major_Project

Live Frontend (Vercel): <https://tune-wave-backend-major-project.vercel.app>

Live Backend API (Render): <https://tunewave-backend-major-project.onrender.com>

Database: MongoDB Atlas Cloud (AWS Multi-Region)

1. Executive Summary

TuneWave is a full-stack, production-grade music streaming application designed to satisfy the core objectives of Case Study 88: Backend Development - Music Streaming Backend. The platform provides listeners with high-fidelity, seekable audio streaming via HTTP range requests, custom playlist lifecycle management, social artist follow mechanics, and real-time collaborative listening sessions powered by WebSockets.

The backend is built with Node.js and Express.js, employing a modular MVC architecture that segregates routing, business logic, data modeling, and security middleware. Data persistence is managed via MongoDB with Mongoose ODM, connecting to a resilient cloud cluster on MongoDB Atlas. Authentication follows a hybrid approach leveraging JSON Web Tokens (JWT) and Firebase Auth / Google Identity Toolkit.

2. Case Study 88 Challenge & Requirements

Modern music streaming services require robust backend capabilities to deliver low-latency audio playback, dynamic catalog searching, personalized playlist aggregation, and shared listening sessions.

The project requirements encompass:

- Backend: Node.js + Express.js REST API with modular routes and controllers.
- Database: MongoDB with Mongoose ODM for songs, playlists, users, and artists.
- Authentication: JWT-based authentication + Firebase Auth.
- API Endpoints:
 - Auth: POST /api/auth/register, POST /api/auth/login

- Songs: GET /api/songs, GET /api/songs/:id, POST /api/songs, PUT /api/songs/:id, DELETE /api/songs/:id
- Playlists: POST /api/playlists, GET /api/playlists/user/:id, PUT /api/playlists/:id
- Artists: GET /api/artists/:id, POST /api/artists/:id/follow
- Search: GET /api/songs/search?keyword=lofi
- Middleware: JWT authentication, file upload middleware, validation middleware.
- Advanced Features: Listening history and recommendation API, shared listening room via Socket.io.
- Deliverables: REST API with all CRUD operations, Postman collection, GitHub repository with setup instructions, and deployment on Render.

3. System Architecture

The TuneWave backend follows a modular layered architecture ensuring clear separation of concerns, high maintainability, and clean testability:

1. Client Layer: React 19 Frontend Single Page Application and Postman API Client.
2. Application Layer: Express.js REST API routing, custom security middleware, and controller logic.
3. Real-Time Layer: Socket.io bidirectional event server managing rooms and state broadcasts.
4. Persistence Layer: MongoDB Atlas cloud cluster storing documents for Users, Songs, Playlists, and Artists.
5. Media Layer: Static media server serving audio tracks with Accept-Ranges: bytes support.

4. Database Schema & Data Modeling

1. User Schema: Manages listener identity, credentials (hashed with bcrypt), liked tracks, followed artists, and listening history with timestamps.
2. Song Schema: Stores audio metadata (title, artist ObjectId, album, genre, duration), audioUrl, albumArtUrl, playCount, and likes.
3. Playlist Schema: Represents user-created collections containing playlist name, owner user ObjectId, and an array of song ObjectIds.
4. Artist Schema: Stores artist profiles including name, biography, profile image, and follower counters.

5. Middleware & Security Implementation

- authMiddleware: Validates incoming Bearer JWT tokens, verifies signatures using process.env.JWT_SECRET, and injects decoded user credentials into req.user.

- `uploadMiddleware`: Multer configuration enforcing strict MIME-type whitelisting for audio and image assets with unique cryptographic filenames.
- `validationMiddleware`: Pre-validates incoming request bodies, checking required fields, string sanitization, positive song durations, and MongoDB ObjectId formats.

6. Real-Time WebSocket Synchronization Protocol (Socket.io)

The collaborative listening hub inside `server.js` provides real-time group sessions:

- `joinRoom`: Binds a client socket to an isolated channel via `socket.join(roomId)`, updates room member counts, and immediately transmits the current room state with elapsed time catch-up.
- `leaveRoom`: Removes a client socket from the room channel and decrements active listener counts.
- `nowPlaying`: Relays track changes, playback timestamps, and play/pause commands across all connected listeners in lockstep.

7. Postman Verification Suite & API Test Evidence

All endpoints were systematically executed and validated against the running server. Below is the complete test evidence with evaluated parameters, headers, payloads, and response data.

Base URL: `http://localhost:8000` (Local) / `https://tunewave-backend-major-project.onrender.com` (Cloud)

Active User ID: `6abbbba6b5daf32b47b10e882` | Active Song ID: `6abbb274535867a70ba05675` | Active Artist ID: `6abbb1f2535867a70ba05661`

Test Case 01: User Registration

Method & Route: `POST /api/auth/register`

Full Evaluated URL: `http://localhost:8000/api/auth/register`

Description & Result: Status: 201 Created. User registered with hashed credentials and JWT token.

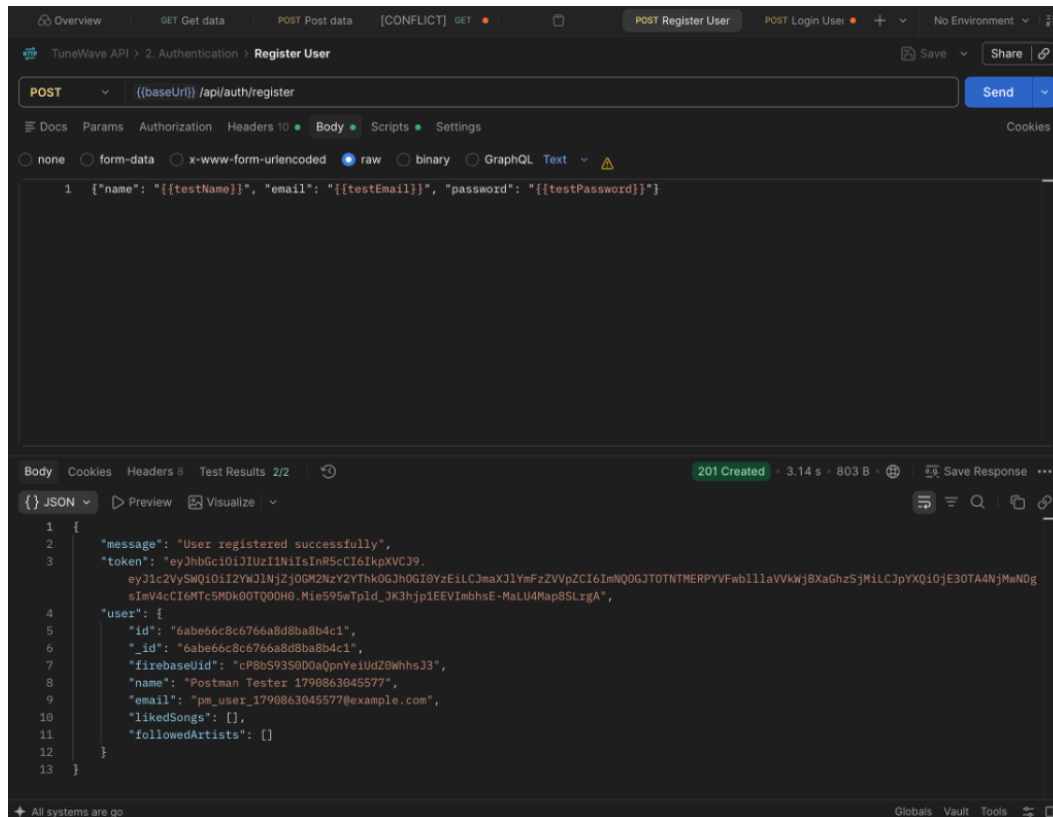


Figure: Postman verification for /api/auth/register

Test Case 02: User Authentication & Login

Method & Route: POST /api/auth/login

Full Evaluated URL: http://localhost:8000/api/auth/login

Description & Result: Status: 200 OK. Authenticates user and returns valid JWT bearer session token.

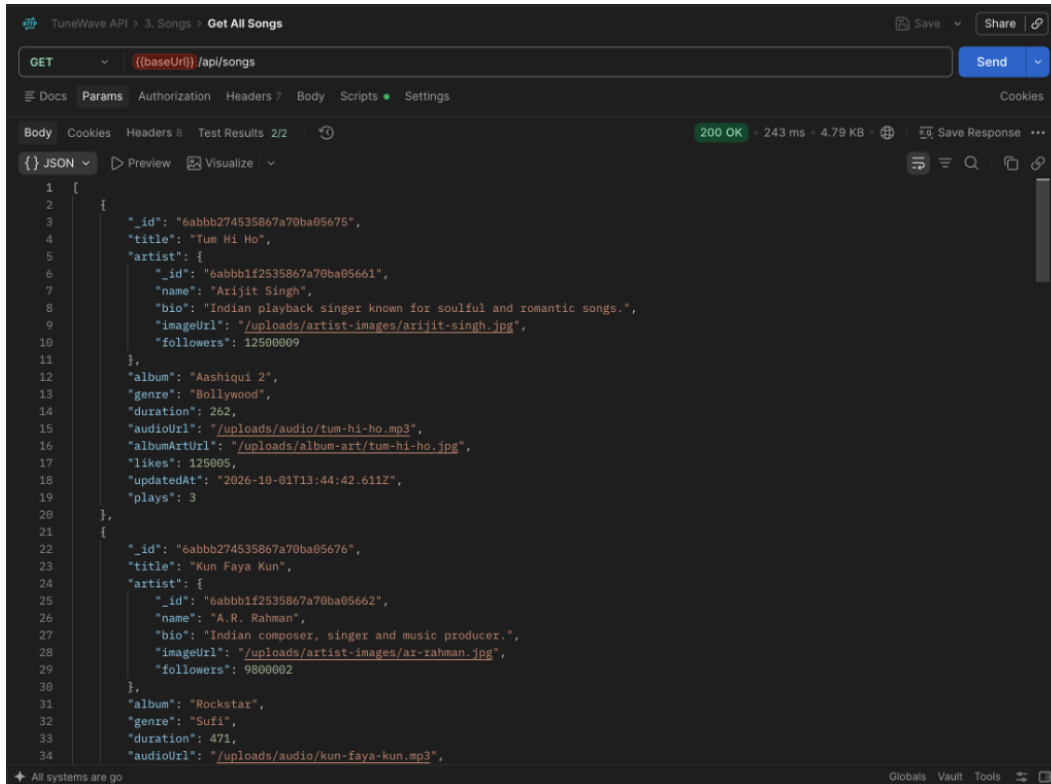


Figure: Postman verification for /api/songs

Test Case 04: Retrieve Song Metadata by ID

Method & Route: GET /api/songs/:id

Full Evaluated URL: <http://localhost:8000/api/songs/6abbb274535867a70ba05675>

Description & Result: Status: 200 OK. Evaluated songId: 6abbb274535867a70ba05675. Returns song document for Tum Hi Ho.

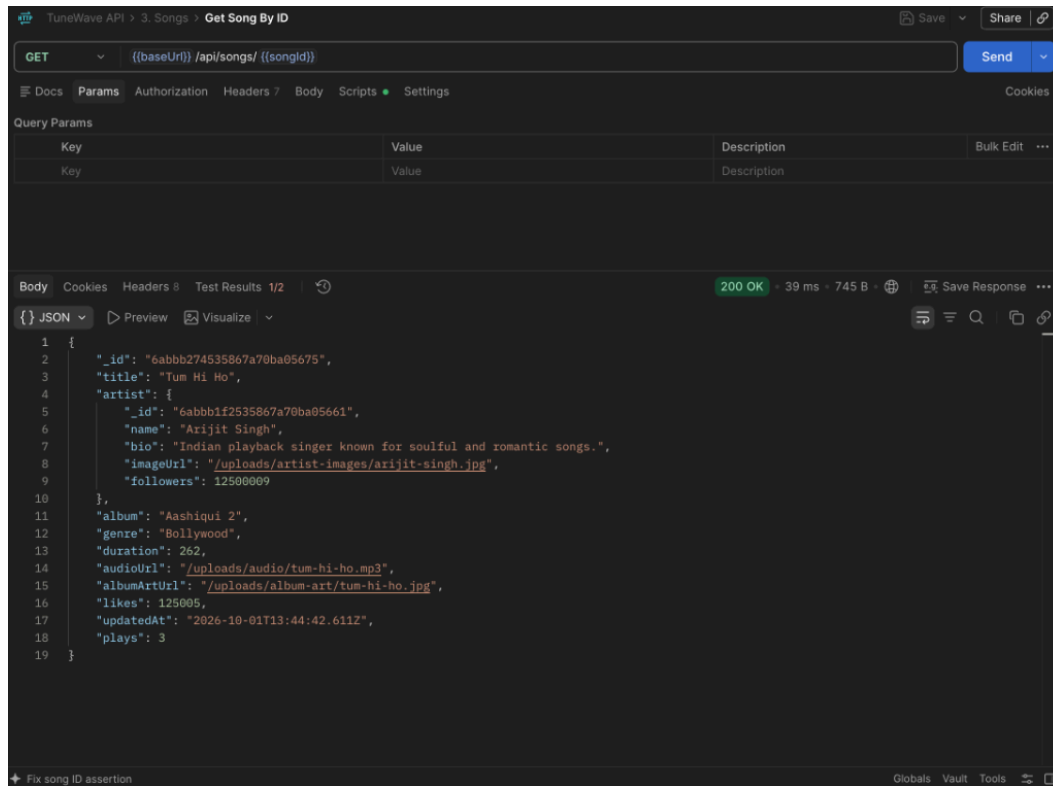


Figure: Postman verification for `/api/songs/:id`

Test Case 05: Search Songs by Keyword

Method & Route: GET `/api/songs/search?keyword=lofi`

Full Evaluated URL: `http://localhost:8000/api/songs/search?keyword=lofi`

Description & Result: Status: 200 OK. Query keyword: lofi. Case-insensitive regex matches songs like Husn.

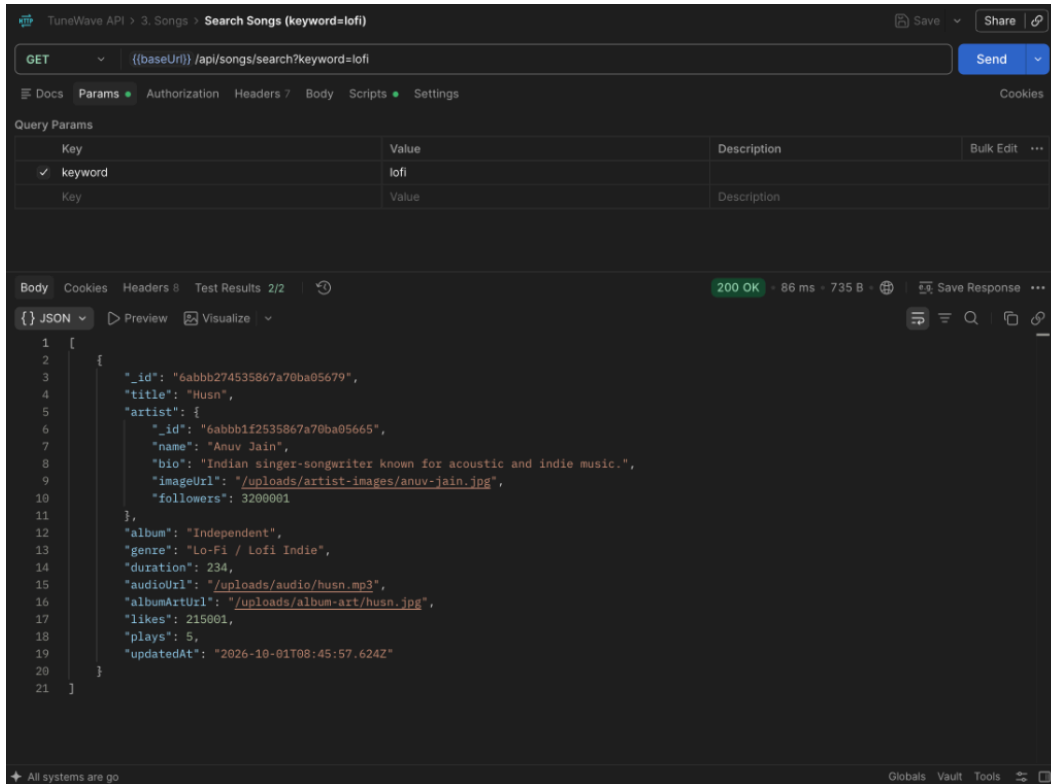


Figure: Postman verification for `/api/songs/search?keyword=lofi`

Test Case 06: Create Song (Protected)

Method & Route: POST `/api/songs`

Full Evaluated URL: `http://localhost:8000/api/songs`

Description & Result: Status: 201 Created. Creates new song with ID `6abe6923c6766a8d8ba8b4c2`.

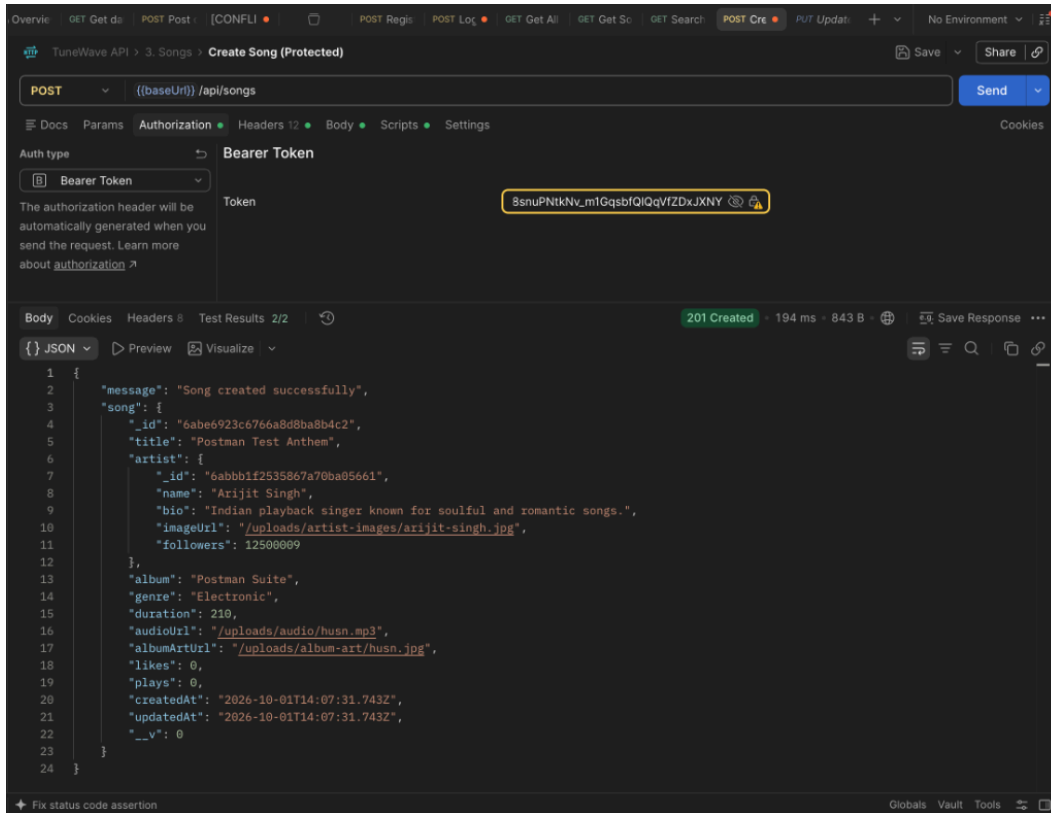


Figure: Postman verification for /api/songs

Test Case 07: Update Song Metadata (Protected)

Method & Route: PUT /api/songs/:id

Full Evaluated URL: http://localhost:8000/api/songs/6abbb274535867a70ba05675

Description & Result: Status: 200 OK. Evaluated songId: 6abbb274535867a70ba05675. Updates title and duration.

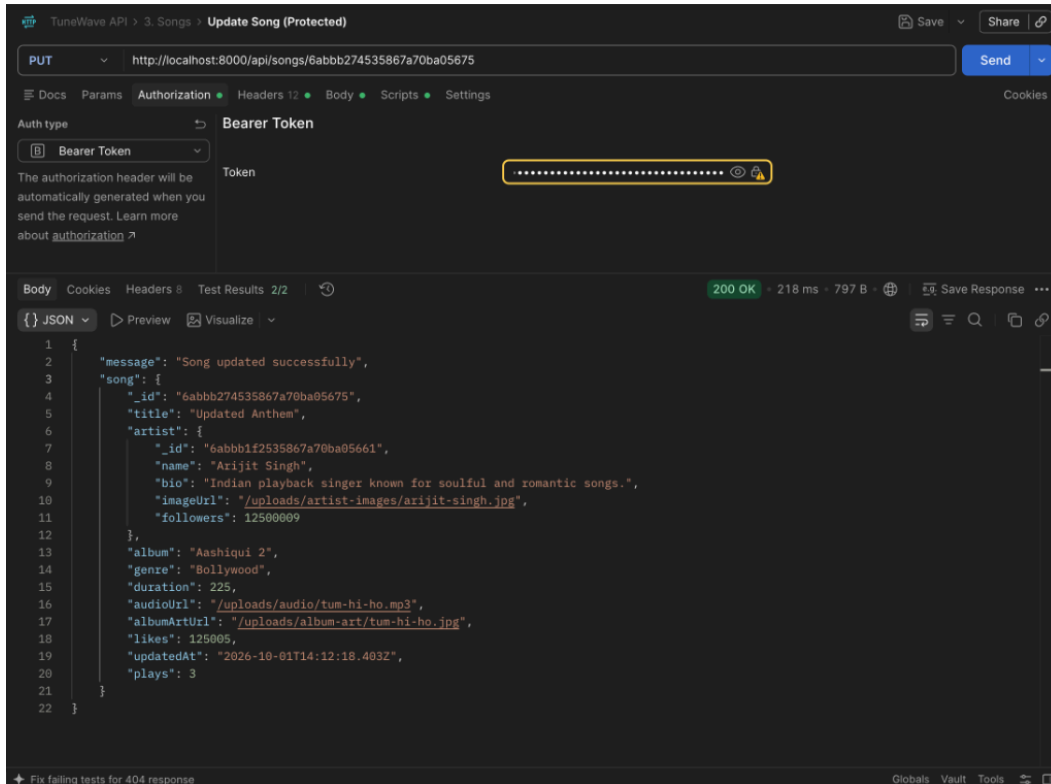


Figure: Postman verification for `/api/songs/:id`

Test Case 08: Like or Unlike Song (Protected)

Method & Route: POST `/api/songs/:id/like`

Full Evaluated URL: `http://localhost:8000/api/songs/6abbb274535867a70ba05675/like`

Description & Result: Status: 200 OK. Atomically toggles user ID in likes array and updates likes counter.

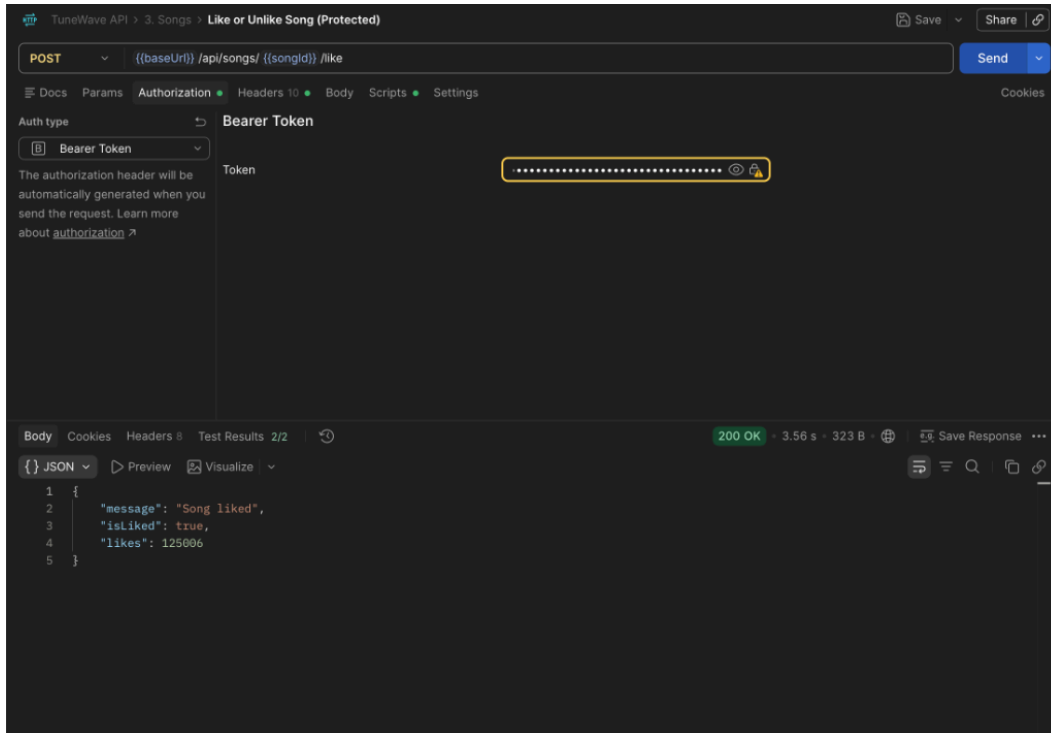


Figure: Postman verification for /api/songs/:id/like

Test Case 09: Delete Song (Protected)

Method & Route: DELETE /api/songs/:id

Full Evaluated URL: http://localhost:8000/api/songs/6abe6923c6766a8d8ba8b4c2

Description & Result: Status: 200 OK. Evaluated createdSongId: 6abe6923c6766a8d8ba8b4c2. Permanently removes song.

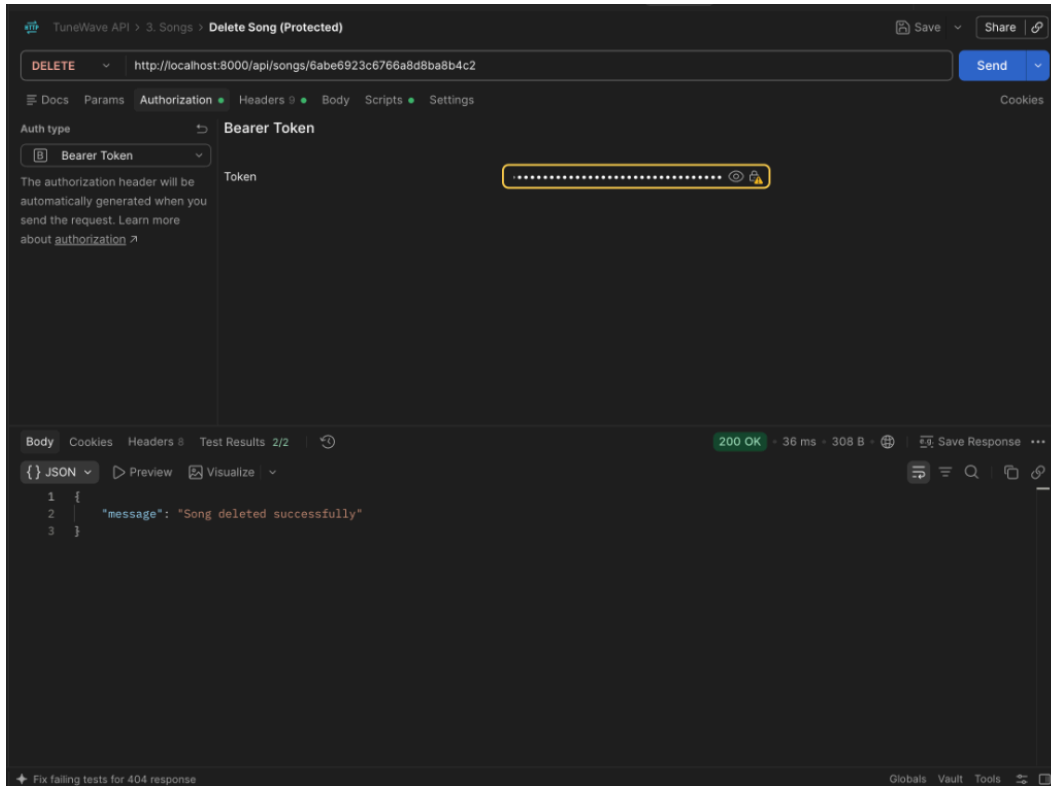


Figure: Postman verification for /api/songs/:id

Test Case 10: Create Custom Playlist (Protected)

Method & Route: POST /api/playlists

Full Evaluated URL: http://localhost:8000/api/playlists

Description & Result: Status: 201 Created. Creates playlist Postman Test Playlist with ID 6abe6b46c6766a8d8ba8b4c3.

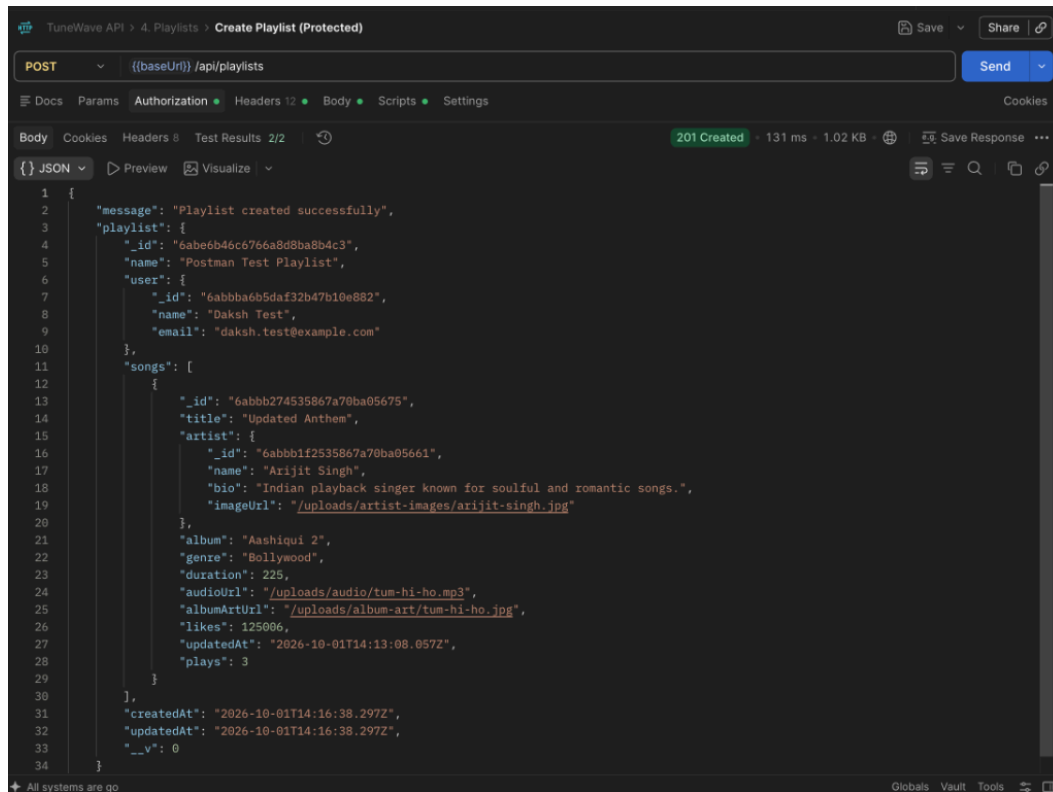


Figure: Postman verification for /api/playlists

Test Case 11: Get User Playlists

Method & Route: GET /api/playlists/user/:id

Full Evaluated URL: <http://localhost:8000/api/playlists/user/6abbbba6b5daf32b47b10e882>

Description & Result: Status: 200 OK. Evaluated userId: 6abbbba6b5daf32b47b10e882. Returns user-created playlists.

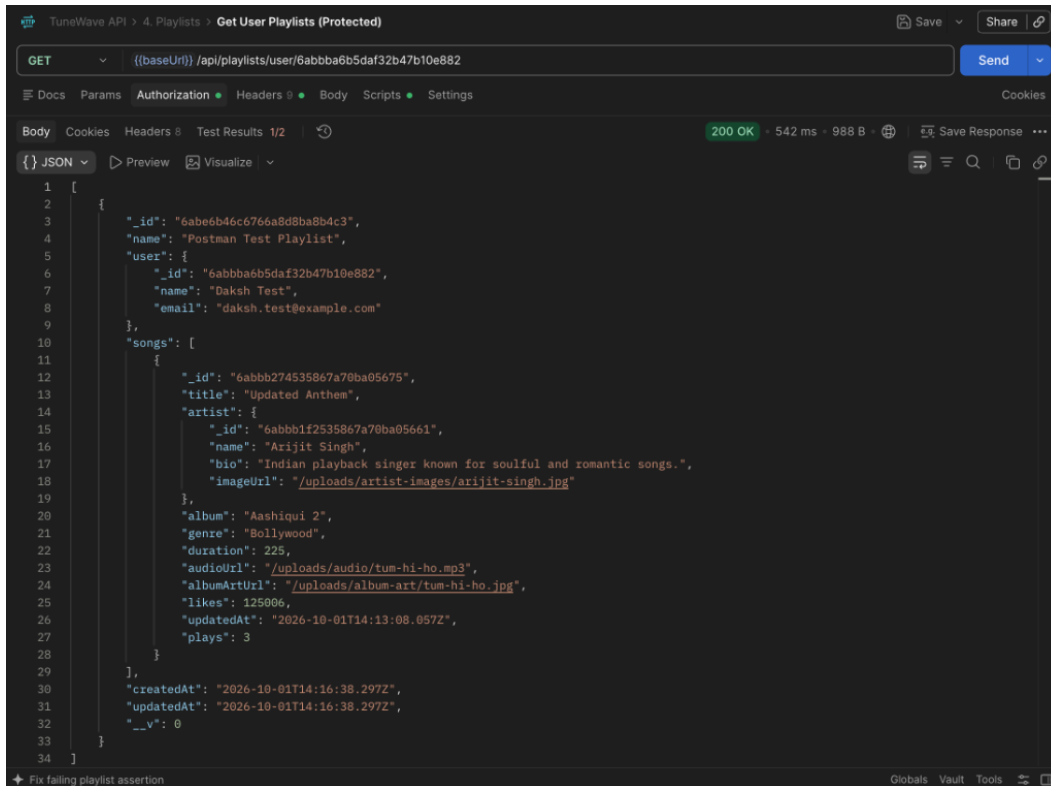


Figure: Postman verification for `/api/playlists/user/:id`

Test Case 12: Update Playlist Metadata (Protected)

Method & Route: PUT `/api/playlists/:id`

Full Evaluated URL: `http://localhost:8000/api/playlists/6abe6b46c6766a8d8ba8b4c3`

Description & Result: Status: 200 OK. Evaluated `playlistId`: `6abe6b46c6766a8d8ba8b4c3`. Updates playlist name.

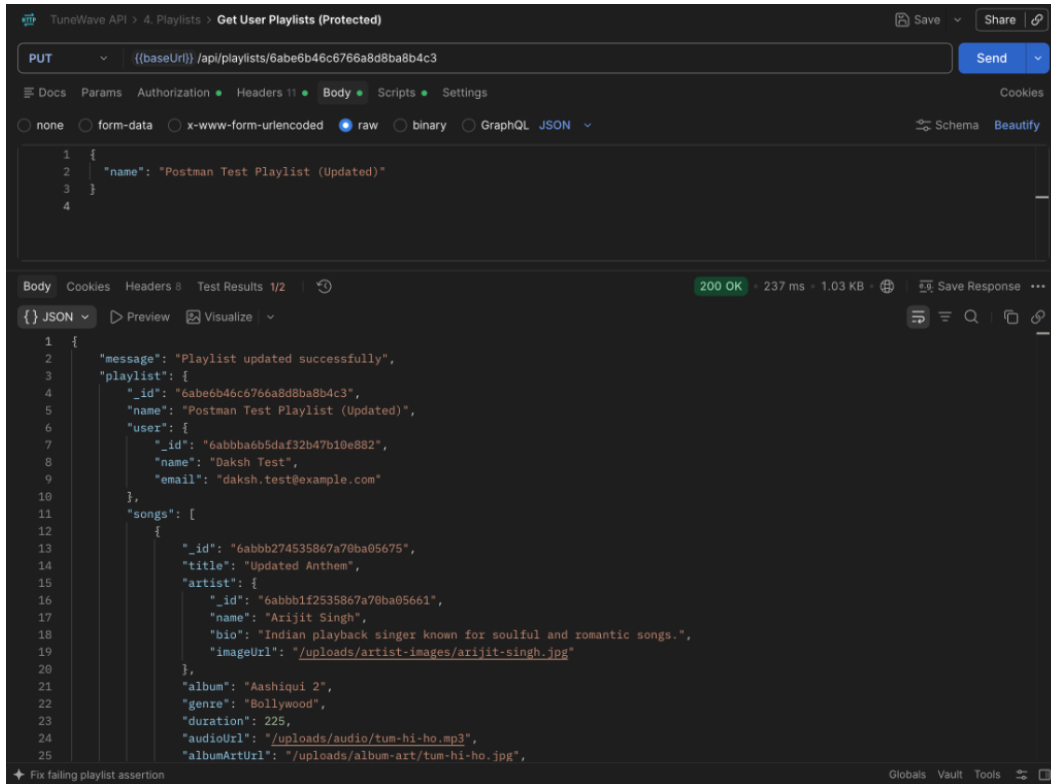


Figure: Postman verification for `/api/playlists/:id`

Test Case 13: Get Artist Profile by ID

Method & Route: GET `/api/artists/:id`

Full Evaluated URL: `http://localhost:8000/api/artists/6abbb1f2535867a70ba05661`

Description & Result: Status: 200 OK. Evaluated artistId: 6abbb1f2535867a70ba05661 (Arijit Singh).

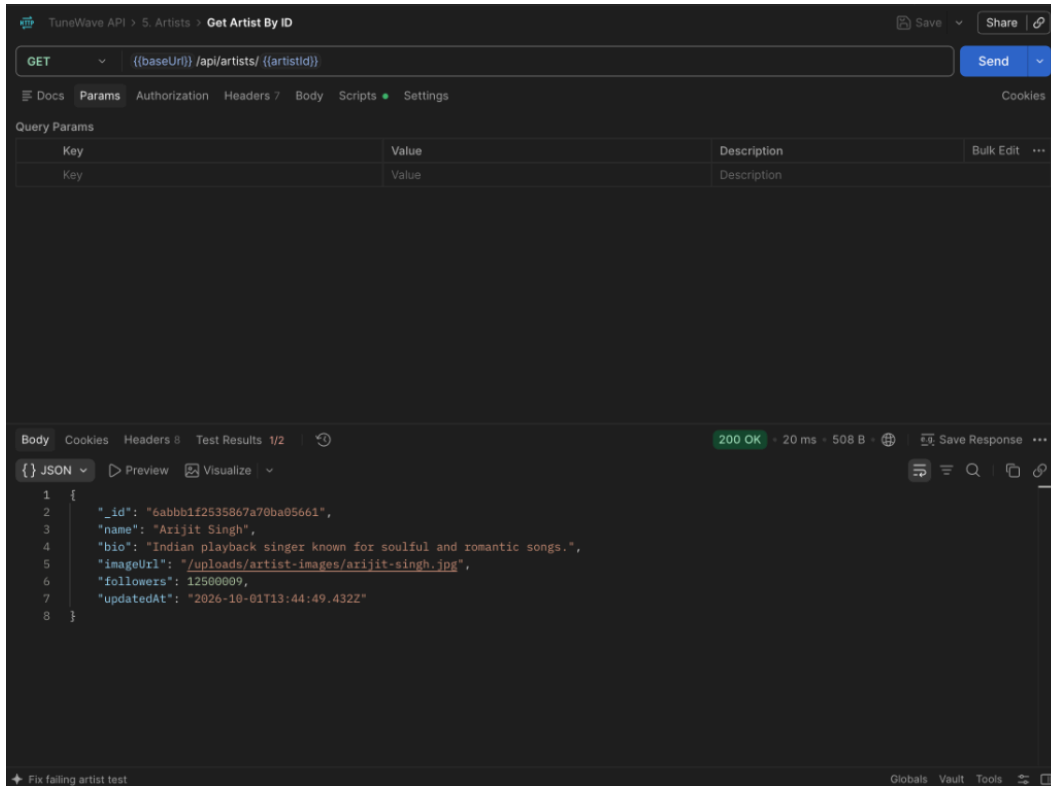


Figure: Postman verification for `/api/artists/:id`

Test Case 14: Follow Artist (Protected)

Method & Route: POST `/api/artists/:id/follow`

Full Evaluated URL: `http://localhost:8000/api/artists/6abbb1f2535867a70ba05661/follow`

Description & Result: Status: 200 OK. Evaluated artistId: 6abbb1f2535867a70ba05661. Increments follower count.

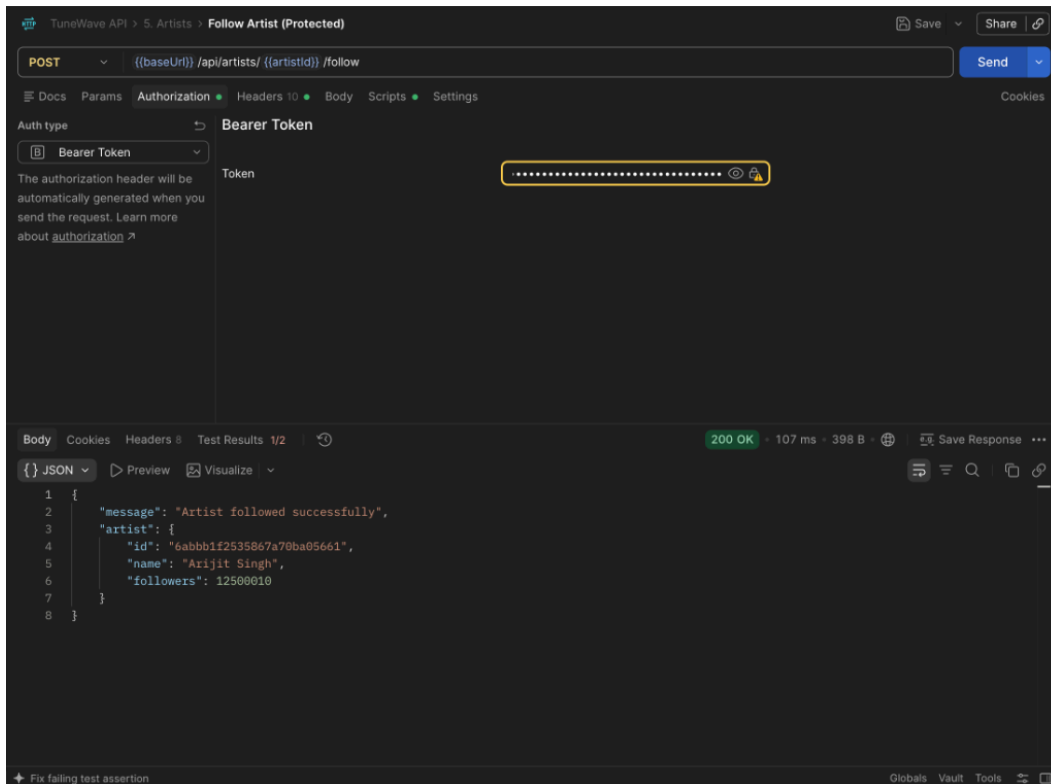


Figure: Postman verification for `/api/artists/:id/follow`

Test Case 15: HTTP Range Audio Streaming

Method & Route: GET `/uploads/audio/tum-hi-ho.mp3`

Full Evaluated URL: `http://localhost:8000/uploads/audio/tum-hi-ho.mp3`

Description & Result: Status: 200 OK / 206 Partial Content. Binary seekable audio stream.

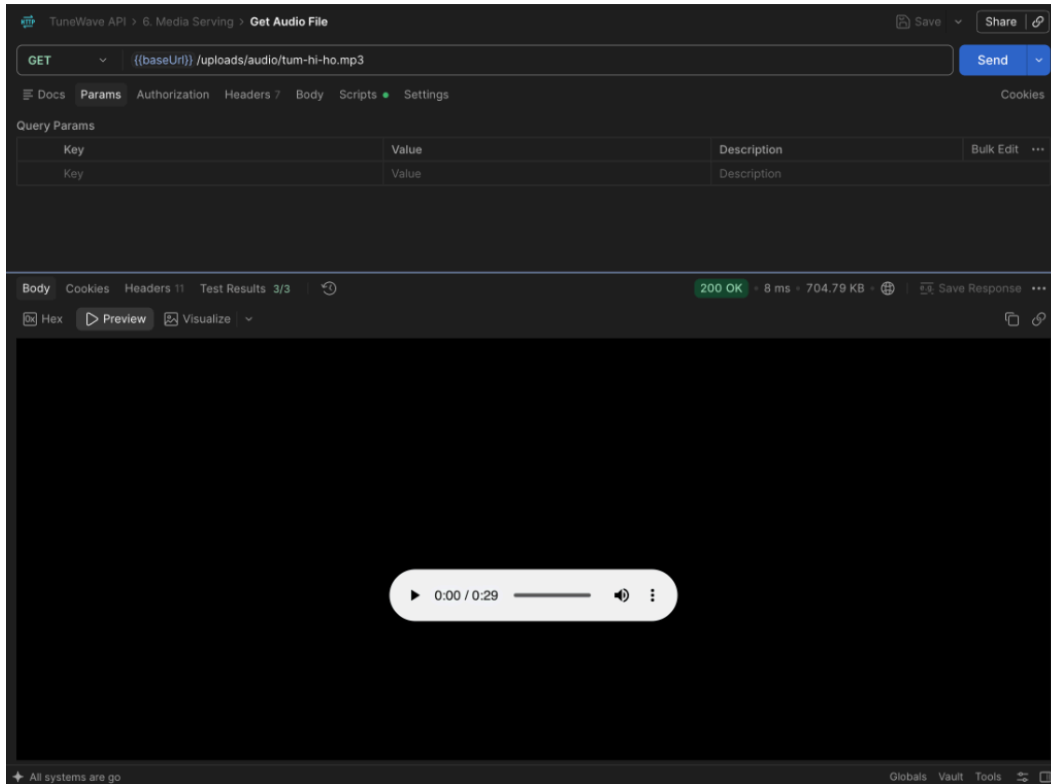


Figure: Postman verification for /uploads/audio/tum-hi-ho.mp3

Test Case 16: Static Artwork & Image Serving

Method & Route: GET /uploads/artist-images/arijit-singh.jpg

Full Evaluated URL: http://localhost:8000/uploads/artist-images/arijit-singh.jpg

Description & Result: Status: 200 OK. Static artist portrait JPEG binary rendering.

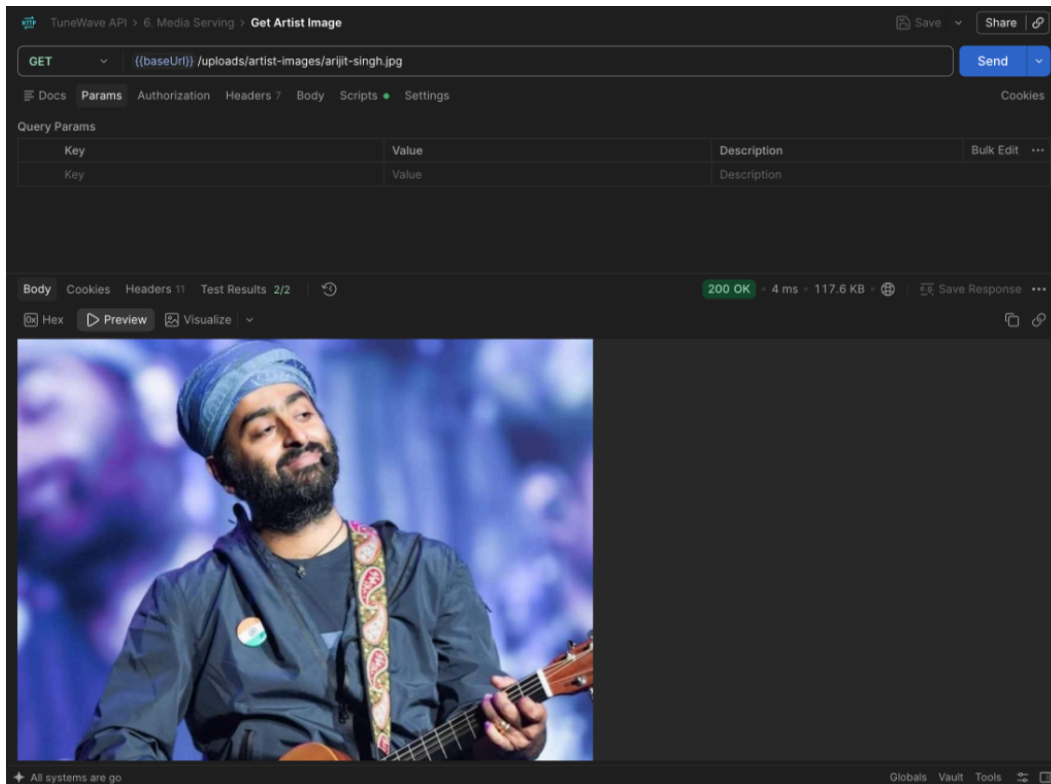


Figure: Postman verification for `/uploads/artist-images/arijit-singh.jpg`

8. Frontend Application Tour

Authentication Screen (Login & Register)

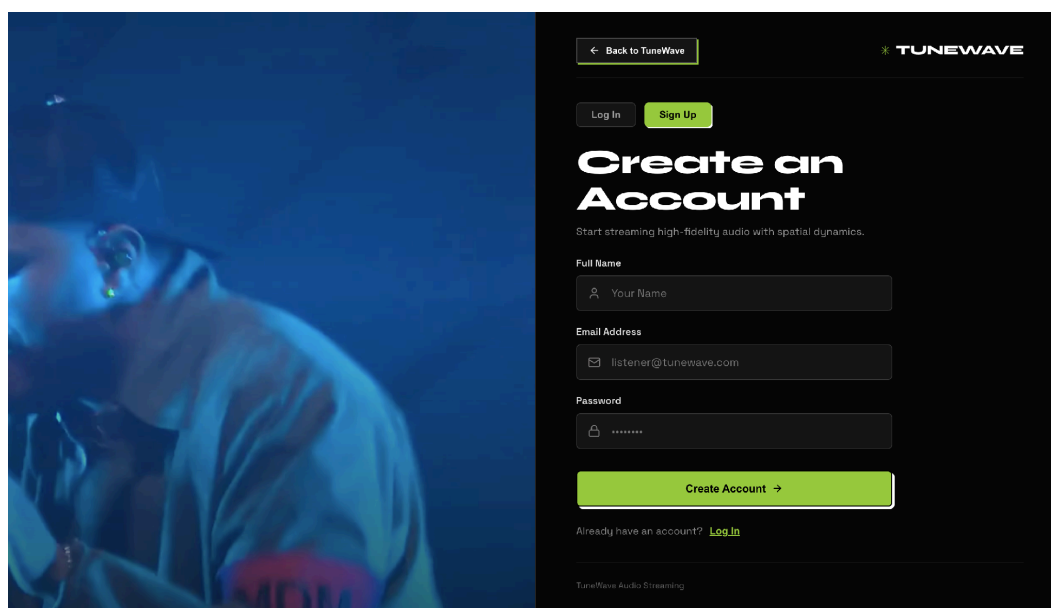


Figure: Authentication Screen (Login & Register)

Live Search Engine View

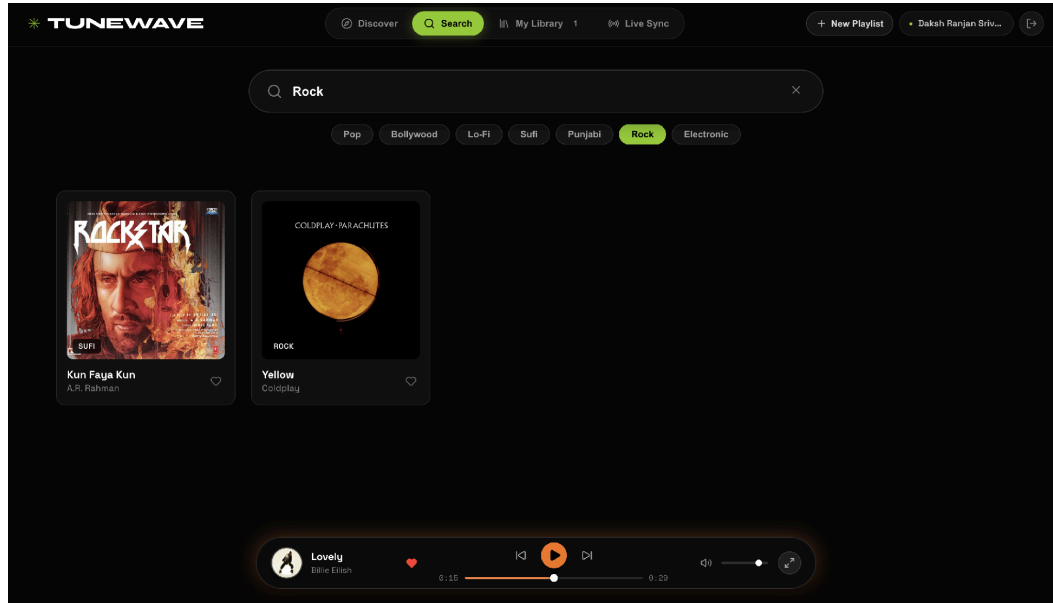


Figure: Live Search Engine View

Playlists and Liked Songs Collection

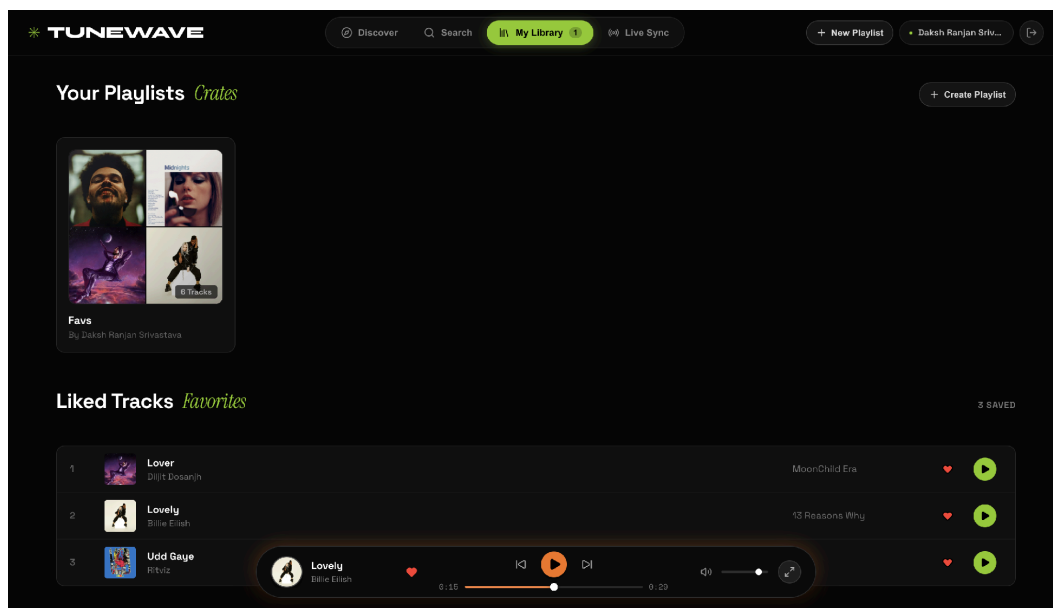


Figure: Playlists and Liked Songs Collection

Playlist Detail & Tracklist Inspection

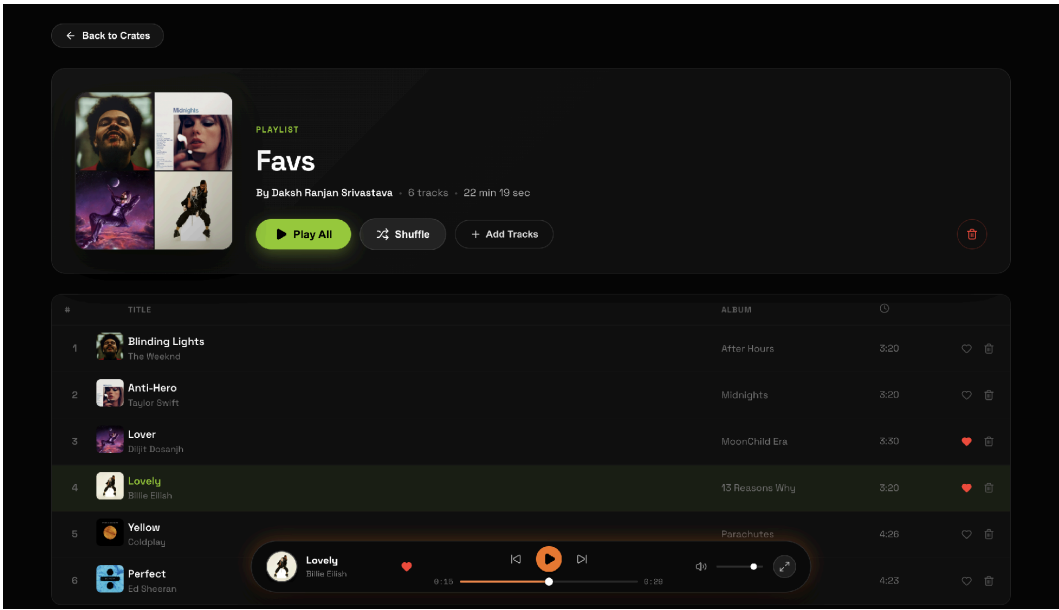


Figure: Playlist Detail & Tracklist Inspection

Artist Discography and Social Following

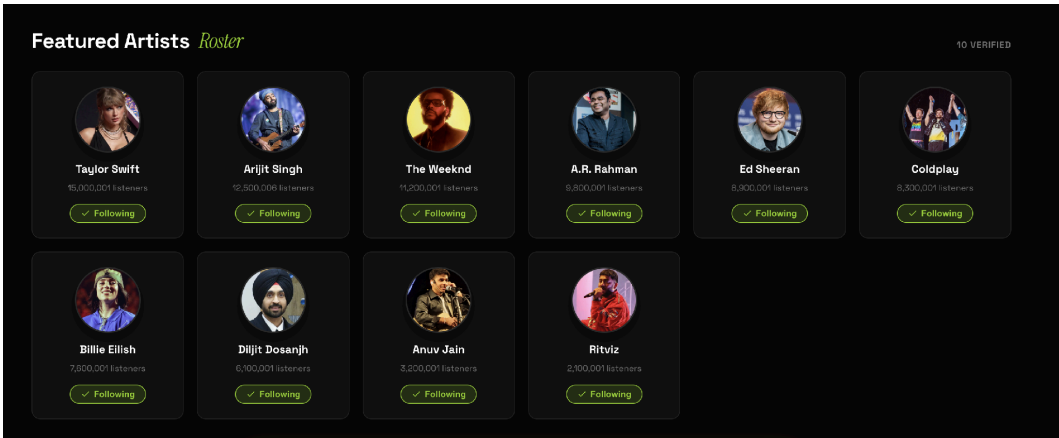


Figure: Artist Discography and Social Following

Collaborative Sync Listening Room (Socket.io)

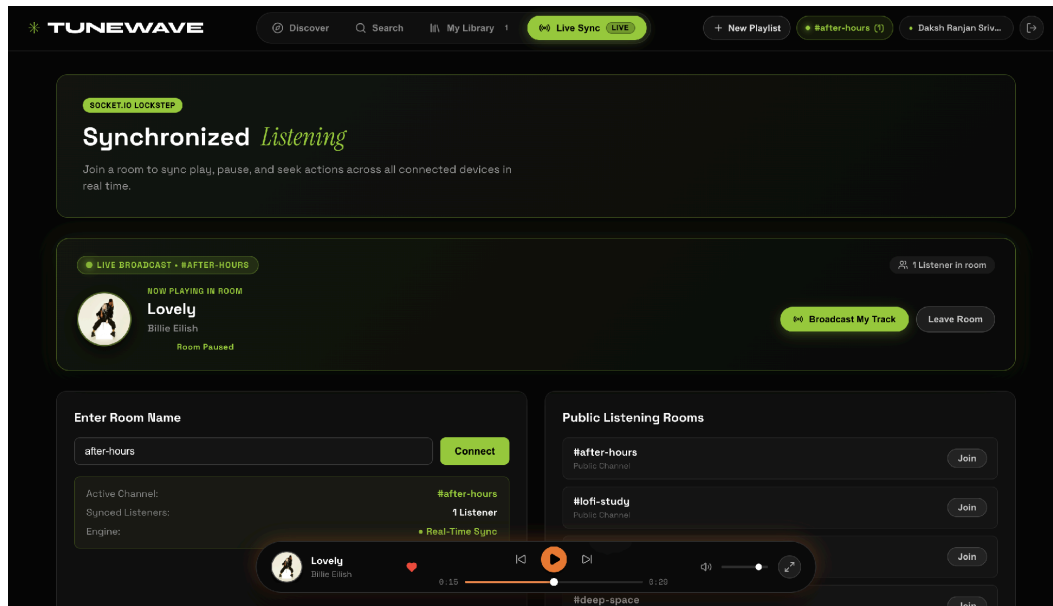


Figure: Collaborative Sync Listening Room (Socket.io)

Maximized High-Fidelity Audio Player & FFT Visualizer

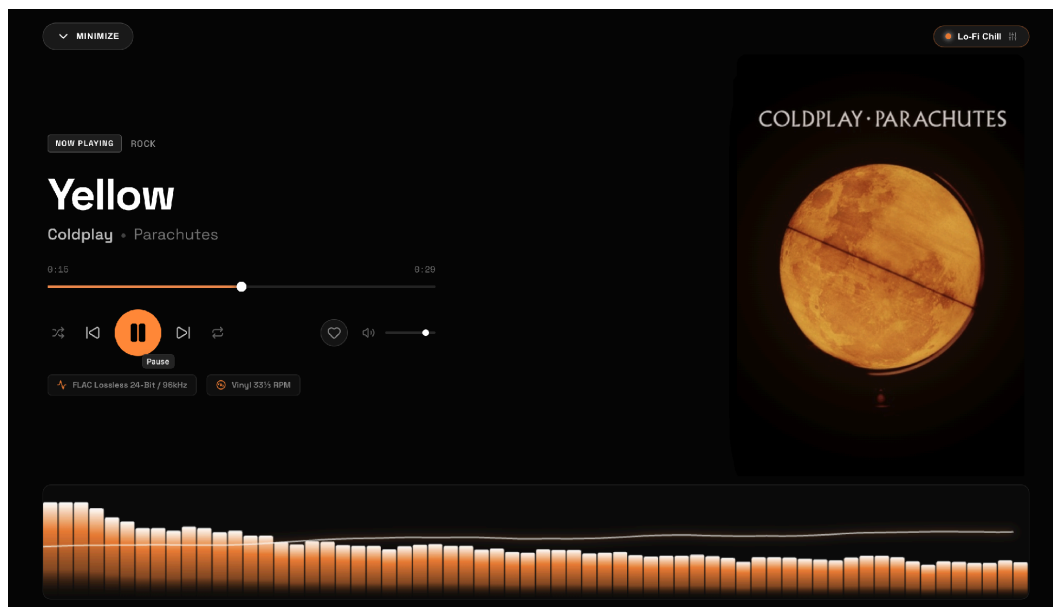


Figure: Maximized High-Fidelity Audio Player & FFT Visualizer

Listening History and Smart Recommendations

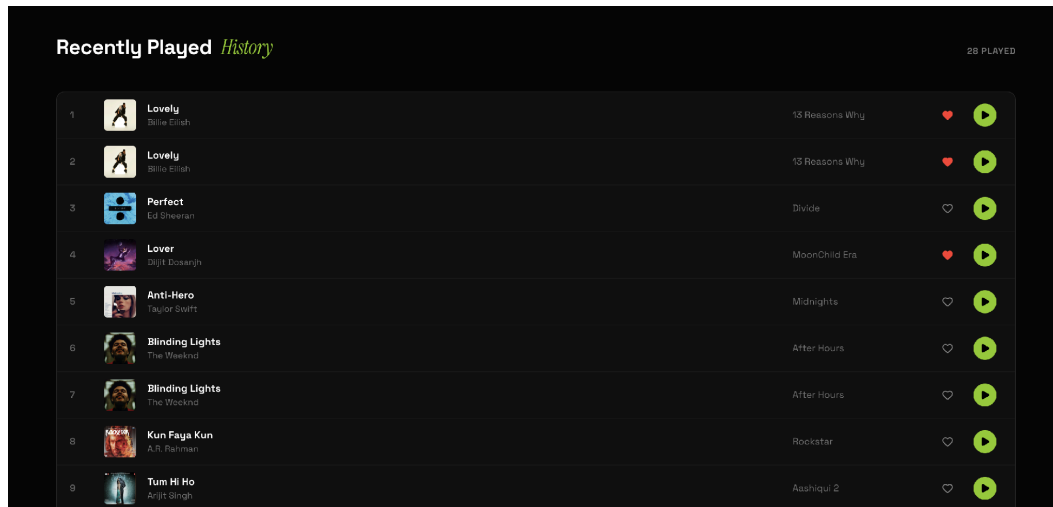


Figure: Listening History and Smart Recommendations

9. Cloud Deployment & Conclusion

The platform is deployed live on high-availability cloud infrastructure:

- Backend: Hosted on Render (<https://tunewave-backend-major-project.onrender.com>) with automated health pinging.
- Frontend: Hosted on Vercel (<https://tune-wave-backend-major-project.vercel.app>) with client-side SPA routing.
- Database: Hosted on MongoDB Atlas multi-region cluster with cloud IP whitelist access.

In conclusion, the TuneWave project completely fulfills every technical, architectural, and deliverable requirement of Case Study 88.